

Memoria di Massa



Memoria di Massa

- Overview della struttura di un Sistema di Memoria di Massa
- Struttura del Disco
- Schedulazione del Disco
- Gestione del disco
- Swap-Space Gestione
- Struttura RAID
- Implementazione Stable-Storage

Obiettivi

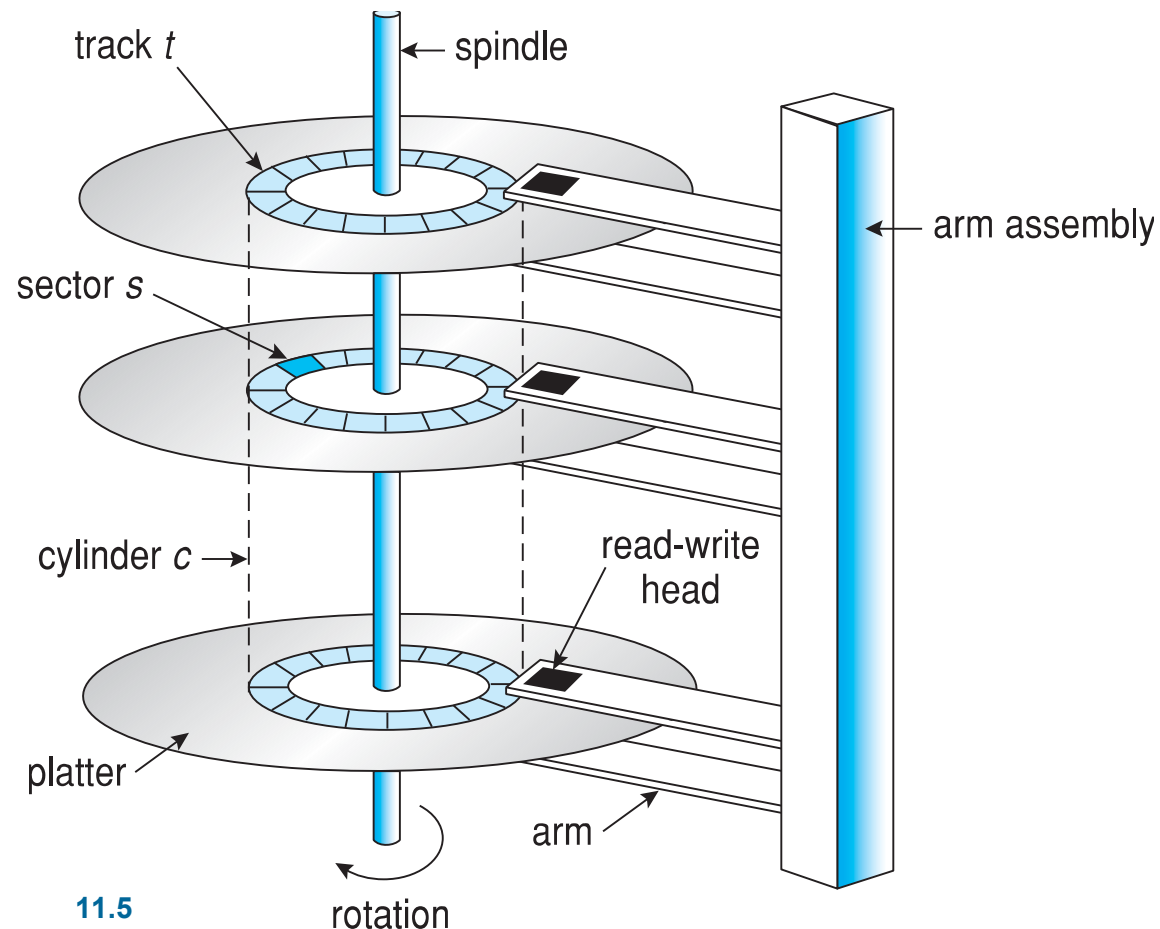
- Descrivere la struttura fisica di un dispositivo di memoria secondario e i suoi effetti sull'uso del dispositivo
- Spiegare le performance dei dispositivi di memoria di massa
- Valutare gli algoritmi di disk scheduling
- Discutere i servizi del SO per la memorizzazione di massa, inclusi i RAID

Overview della Struttura di Memoria di Massa

- Consideriamo due tipi di memorie secondarie
 - **Hard Disk Drives (HDDs)** e **NonVolatile Memory (NVM)**
 - Coprono la maggior parte dei dispositivi per la memoria secondaria dei moderni computer
 - Si descrivono le caratteristiche di questi dispositivi

Hard Disk

- ❑ Strutturati su dischi
- ❑ Dischi piatti come CD hanno dimensioni da .85" a 14" (storicamente)
 - ❑ Comunemente 3.5, 2.5 e 1.8 pollici
 - ❑ Superfici coperte da materiale magnetico
- ❑ Una testina read-write si muove sui piatti
- ❑ Braccio muove le testine
- ❑ Piatti divisi in tracce
- ❑ Tracce divise in settori
- ❑ Testine si muovono su cilindri (insieme di tracce)
- ❑ Migliaia di cilindri, per ogni traccia centinaia di settori
- ❑ Prima settori 512 bytes ora fino a 4 KB



Hard Disk

- ❑ Capacità da GB a TB
- ❑ Motore disco
 - ❑ da 60 a 250 rotazioni per secondo,
 - ❑ rotation per minute (RPM) da 5400 a 15000
- ❑ Performance
 - ❑ Transfer Rate
 - ▶ da disco a computer
 - ▶ teorico – 6 Gb/sec
 - ▶ Effective Transfer Rate – real – 1Gb/sec
 - ❑ Tempo di posizionamento
 - ▶ Seek time e rotational latency
 - ▶ Seek time da 3ms a 12ms – 9ms comune per desktop drives
 - ▶ Latenza dipende da spindle speed
 - ▶ $1 / (\text{RPM} / 60) = 60 / \text{RPM}$
 - ▶ Average latency = $\frac{1}{2}$ latency

Spindle [rpm]	Average latency [ms]
4200	7.14
5400	5.56
7200	4.17
10000	3
15000	2



Hard Disk Performance

- **Access Latency** = **Average access time** = average seek time + average latency
 - Per dischi più veloci $3\text{ms} + 2\text{ms} = 5\text{ms}$
 - Per dischi lenti $9\text{ms} + 5.56\text{ms} = 14.56\text{ms}$

- Average I/O time = average access time + (quantità da trasferire / velocità di trasferimento) + overhead del controllore

- Per esempio per trasferire un blocco di 4KB su un disco da 7200 RPM con un average seek time di 5ms, 1Gb/sec di transfer rate con un .1ms overhead del controllore =
 - $5\text{ms} + 4.17\text{ms} + 0.1\text{ms} + \text{transfer time} =$
 - Transfer time = $4\text{KB} / 1\text{Gb/s} = 0.031\text{ ms}$
 - Average I/O time for 4KB block = $9.27\text{ms} + .031\text{ms} = 9.301\text{ms}$

Il Primo Disk Drive Commerciale



1956
IBM RAMDAC computer
included the IBM Model
350 disk storage system

5M (7 bit) characters
50 x 24" platters
Access time = < 1 second

Dispositivi NVM

I dispositivi **NVM** sempre più rilevanti e diffusi. Sono elettrici e non meccanici. Tipicamente composti basati su memoria flash (chip NAND flash) vengono spesso inseriti in contenitori simili a unità disco, e per questa ragione sono chiamati **dischi a stato solido**, o **SSD**

Un dispositivo NVM può anche assumere la forma di un'**unità USB**

In tutte le sue forme possiamo trattarlo in modo uniforme



Figura 11.3 Scheda di un SSD da 3,5 pollici.

Solid-State Disks (SSDs)

- ❑ Memoria nonvolatile usata come un hard drive
 - ❑ Non meccanica
 - ❑ Implementata con diverse tecnologie
- ❑ Può essere più affidabile di HDDs
- ❑ Più costosa per MB
- ❑ Forse life span più breve
- ❑ In generale ha meno capacità di HDD ma più veloce
- ❑ Capacità cresce rapidamante ed il prezzo cala, quindi si stanno affermando
- ❑ I bus di connessione possono essere troppo lenti
 - ❑ connessi direttamente al bus di sistema
- ❑ Non parti mobili, non parti meccaniche
 - ❑ **non seek time e rotational latency**

Algoritmi del controllore delle NAND Flash

Implementate con **semiconduttori NAND** (transistor MOSFET) che portano nuove sfide a capacità di memorizzazione e affidabilità

I **semiconduttori NAND** *non possono essere sovrascritti direttamente*, devono prima essere cancellati. Organizzati in pagine e presentano *pagine* che contengono dati *non validi*.

La cancellazione avviene in blocco e prende tempo ($>$ del tempo di scrittura $>$ tempo lettura). Le operazioni possono avvenire in parallelo. Deterioramento dopo ogni cancellazione. Durata misurata in Drive Writes per Day: quanto volte può essere scritta per giorno prima del fallimento (es. 1 TB può avere 5 DWPD nel periodo garantito)

Algoritmi gestiti dal controller, non dal sistema Operativo

pagina valida	pagina valida	pagina non valida	pagina non valida
pagina non valida	pagina valida	pagina non valida	pagina valida

Figura 11.4 Blocco NAND con pagine valide e non valide.

Controllore delle NAND Flash

I **semiconduttori NAND** non possono essere sovrascritti direttamente, devono prima essere cancellati. Sono di solito presenti *pagine* che contengono dati *non validi*.

Il controller contiene una tabella: Flash Translation Layer (FTL) indica quali pagine contengono dati validi

Per gestire i dati occorrono: algoritmi di garbage collection per liberare blocchi da pagine valide, pagine (circa 20%) sempre disponibili per fare il write (**overprovisioning**), meccanismi per distribuire le cancellazioni, meccanismi di correzione del dato (se errori continui su una pagina, questa è marcata come bad)

pagina valida	pagina valida	pagina non valida	pagina non valida
pagina non valida	pagina valida	pagina non valida	pagina valida

Figura 11.4 Blocco NAND con pagine valide e non valide.

Memoria Volatile

- ❑ Memoria volatile (DRAM) può essere usata per fare storage
- ❑ Si parla di RAM drivers o **RAM disk**
- ❑ Creati dai device driver che presentano porzioni della DRAM come se fosse memoria di massa (temporaneo)
- ❑ Accessibile ad utenti e programmatori
- ❑ In Linux /dev/ram e /tmp
- ❑ Memorizzazione temporanea, ma accesso molto rapido

Dischi Magnetici

- Era il primo mezzo di memorizzazione secondaria
 - Evoluto da bobina a cartucce
- Relativamente permanente, contiene larghe quantità di dati
- Tempo di accesso molto lento
- Random access ~1000 volte più lento del disco, 100000 più lento di SSD
- Usate soprattutto per backup, stoccaggio di dati non usati frequentemente, trasferimento tra sistemi
- Mantenuto in bobina e avvolto o riavvolto per la testina di lettura-scrittura
- Una volta che i dati sono sotto testina, velocità di trasferimento simile al disco
 - 140MB/sec e maggiore
- TB di storage

Metodi di Connessione Memoria Secondaria

- Drive connesso al computer via **I/O bus** o il **bus di sistema**
 - Bus variano, es. **EIDE**, **ATA**, **SATA**, **USB**, **Fibre Channel**, **SCSI**, **SAS**, **Firewire**
 - ▶ Per HDDs più comune è il SATA
 - ▶ Per NVM interfacce veloci NVM express che collega direttamente al Peripheral Component Interconnect (PCI) bus
 - Trasferimento dati sul bus gestiti da **controller**
 - **Host controller** gestisce il trasferimento dati lato computer che usa il bus per parlare con il **disk controller**

Mapping degli Indirizzi

- Le unità disco sono indirizzati come grandi array 1-dimensionali di **blocchi logici** dove il logical block è la più piccola unità di trasferimento
 - Formattazione di basso livello creano **blocchi logici** su mezzi fisici
 - Ogni blocco logico mappato sui settori fisici o su pagine di un dispositivo NVM
- Su Disco i blocchi logici sono mappati sequenzialmente in settori
 - Settore 0 è il primo settore della prima traccia sul cilindro più esterno
 - Il mapping procede in ordine, dalla prima al resto delle tracce su quel cilindro e poi attraverso il resto dei cilindri (dal più esterno al più interno)
- Per NVM mapping da tuple (chip, blocco, pagina) a blocchi logici
- Indirizzamento basato su Logical Block Address (LBA) più semplice di settore, cilindro, testina o chip, blocco, pagine

Mapping degli Indirizzi

- Le unità disco sono indirizzati come grandi array 1-dimensionali di **blocchi logici** dove il logical block è la più piccola unità di trasferimento
 - Formattazione di basso livello creano **blocchi logici** su mezzi fisici
 - Ogni blocco logico mappato su un settore fisico o su pagina di un dispositivo NVM

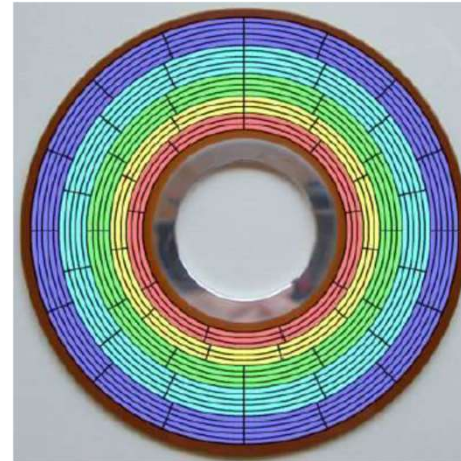
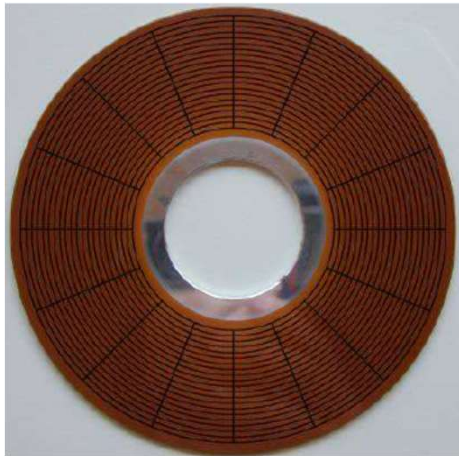
- Su Disco blocchi logici mappati sequenzialmente in settori
 - Unità minima di lettura/scrittura (si scrive in blocchi)
 - Settore di solito 512 bytes individuato mediante un numero a partire da 1
 - Settori per traccia 63
 - Settore individuato da numero della testina, il numero di cilindro e quello del settore CHS (Cylinder Head Sector)
 - LBA (Linear Block Addressing) i settori vengono numerati utilizzando la successione 0,1,2.... Es. primi 63 (C=0,H=0), poi 64 (C=1, H=0, S=1)

Mapping degli Indirizzi

- Le unità disco sono indirizzati come grandi array 1-dimensionali di **blocchi logici** dove il logical block è la più piccola unità di trasferimento
 - Formattazione di basso livello creano **blocchi logici** su mezzi fisici
 - Ogni blocco logico mappato su un settore fisico o su pagina di un dispositivo NVM
- Su Disco blocchi logici mappati sequenzialmente in settori
- Per NVM mapping da una tupla (chip, blocco, pagina) ad un blocco logico
- Il passaggio da indirizzi logici a fisici dovrebbe essere facile ma ...
 - ▶ Settori possono essere danneggiati danneggiati
 - ▶ Non-costante numero di settori per traccia
 - ▶ Gestione interna del mapping tra LBA e settori fisici
 - ▶ Si tende a mantenere comunque allineati indirizzi logici e fisici (salgono insieme)

Struttura Disco

- L'array 1-dimensionale di blocchi logici è mappato in settori del disco sequenzialmente
- Due tipi di organizzazione del disco:



- Stesso numero di settori per traccia
 - Ampiezza variabile del settore e densità scrittura variabile
 - Velocità lettura costante per traccia
 - Tracce esterne poco utilizzate
- Numero di settori differente per traccia (zone bit recording)
 - Densità scrittura costante
 - Velocità lettura settori variabile
 - Ottimizzazione (più dati nei settori esterni)

Struttura Disco

- L'array 1-dimensionale di blocchi logici è mappato in settori del disco sequenzialmente
 - Nei CD velocità aumenta verso l'interno per mantenere la lettura dei settori costante
 - ▶ Constant Linear Velocity (CLV)
 - Nei dischi
 - ▶ Constant Angular Velocity (CAV)
 - ▶ Se stesso numero di settori per traccia densità dei bit varia per mantenere costante il flusso dei dati
 - ▶ In zone bit recording il flusso varia per traccia (velocità di lettura diversa per zona)
- Dimensioni tipiche:
 - ▶ Diverse centinaia di settori per traccia,
 - ▶ Decine di migliaia di cilindri

Scheduling del Disco

- Il SO è responsabile dell'uso efficiente dell'hardware
 - Per le unità disco significa veloce accesso e alto trasferimento dati
- Per le unità a disco
 - minimizzare il seek time e il latency time
 - Seek time $\approx$ seek distance
 - Latency time tempo per arrivare al settore con rotazione
- La disk **bandwidth** è il numero totale di bytes trasferiti diviso per il tempo totale tra la prima richiesta di servizio ed il completamento dell'ultimo trasferimento
- Si può aumentare sia l'accesso che il trasferimento gestendo l'ordine con cui vengono servite le diverse richieste di I/O

Scheduling del Disco

- Ci sono tipi di richieste di I/O da disco
 - Processi di sistema
 - Processi utente
 - Le richieste riguardano input o output, file da aprire, pagine da trasferire, etc.
- SO mantiene una coda di richieste, per disco o dispositivo
- Un disco disponibile in idle può subito servire la richiesta di I/O, il disco occupato gestisce una coda di richieste
 - Gli algoritmi di ottimizzazione hanno un ruolo nel caso di code
 - Nel passato molto sforzo per definire interfacce ai dischi per ottimizzare
 - Oggi molto è gestito direttamente dai controllori del disco
 - ▶ Non è accessibile la locazione esatta della testina di lettura dei dischi
 - ▶ Però si può assumere prossimità tra indirizzi fisici e logici

Scheduling del Disco

- Nota che i controllori di unità hanno piccoli buffer e possono gestire code di richieste di I/O
- Molti algoritmi proposti per schedulare il modo in cui sono servite le richieste di I/O (basati su riduzione del seek time)
- L'analisi è valida per uno o più piatti
- Illustriamo gli algoritmi di scheduling con una coda di richieste a blocchi su cilindri (0-199)

98, 183, 37, 122, 14, 124, 65, 67

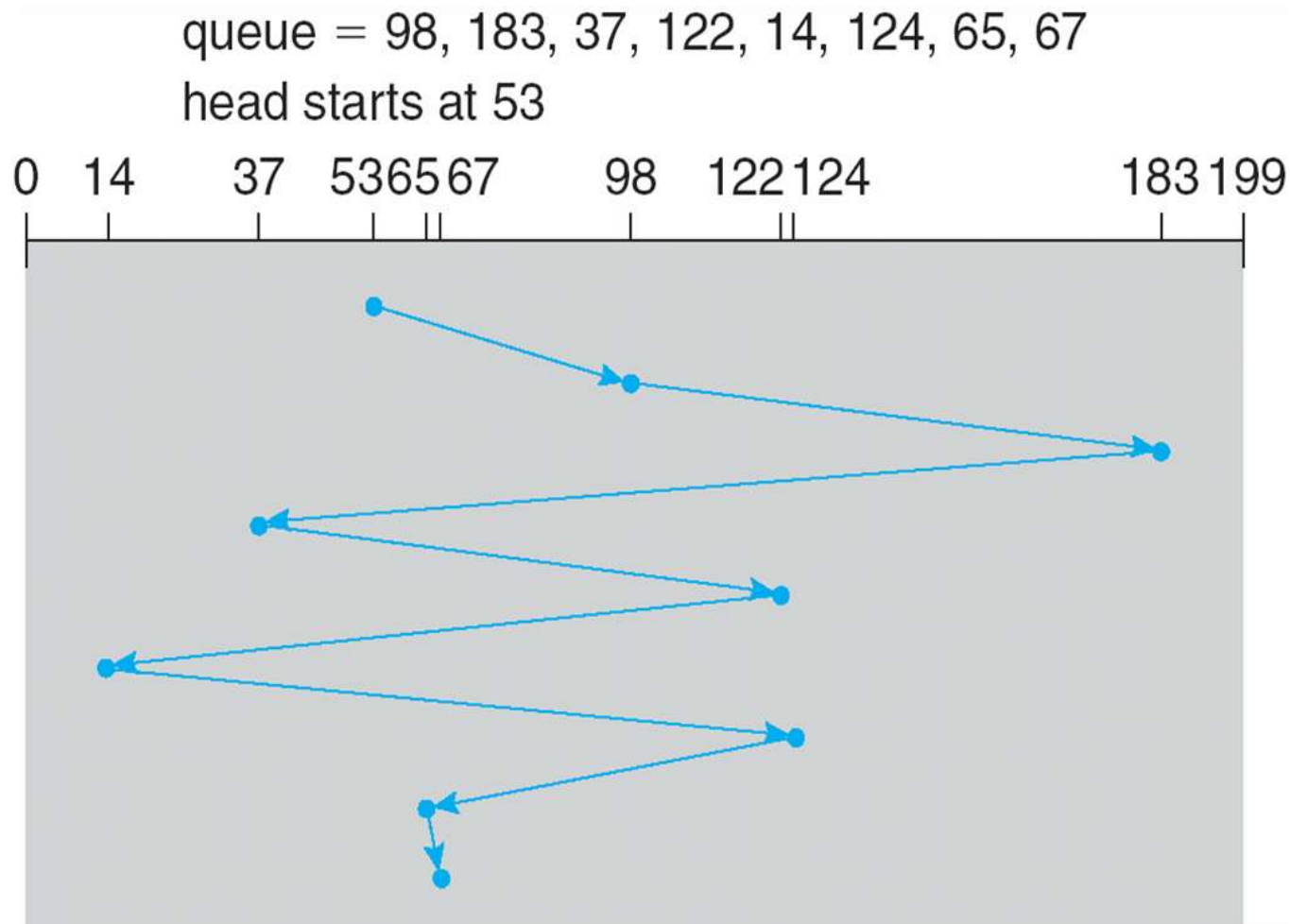
Testina che punta a 53

FCFS

Algoritmo più semplice

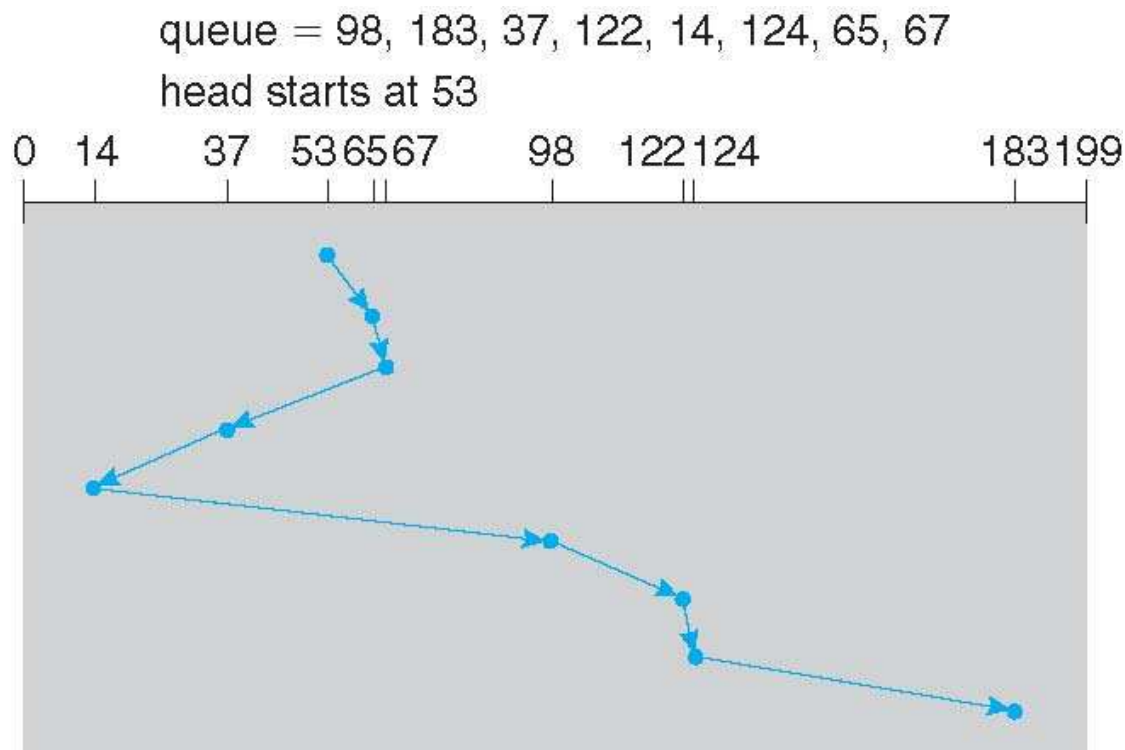
La figura mostra un movimento di testina attraverso 640 cilindri

Si potrebbe migliorare con 37 e 14 serviti vicini, come 122 e 124



SSTF

- ❑ Shortest Seek Time First, seleziona la richiesta con il minimo seek time dalla corrente posizione della testina
- ❑ SSTF scheduling è una forma di SJF scheduling; può causare starvation di qualcuna delle richieste
- ❑ La figura mostra un numero totale di movimenti di testa di 236 cilindri



SCAN

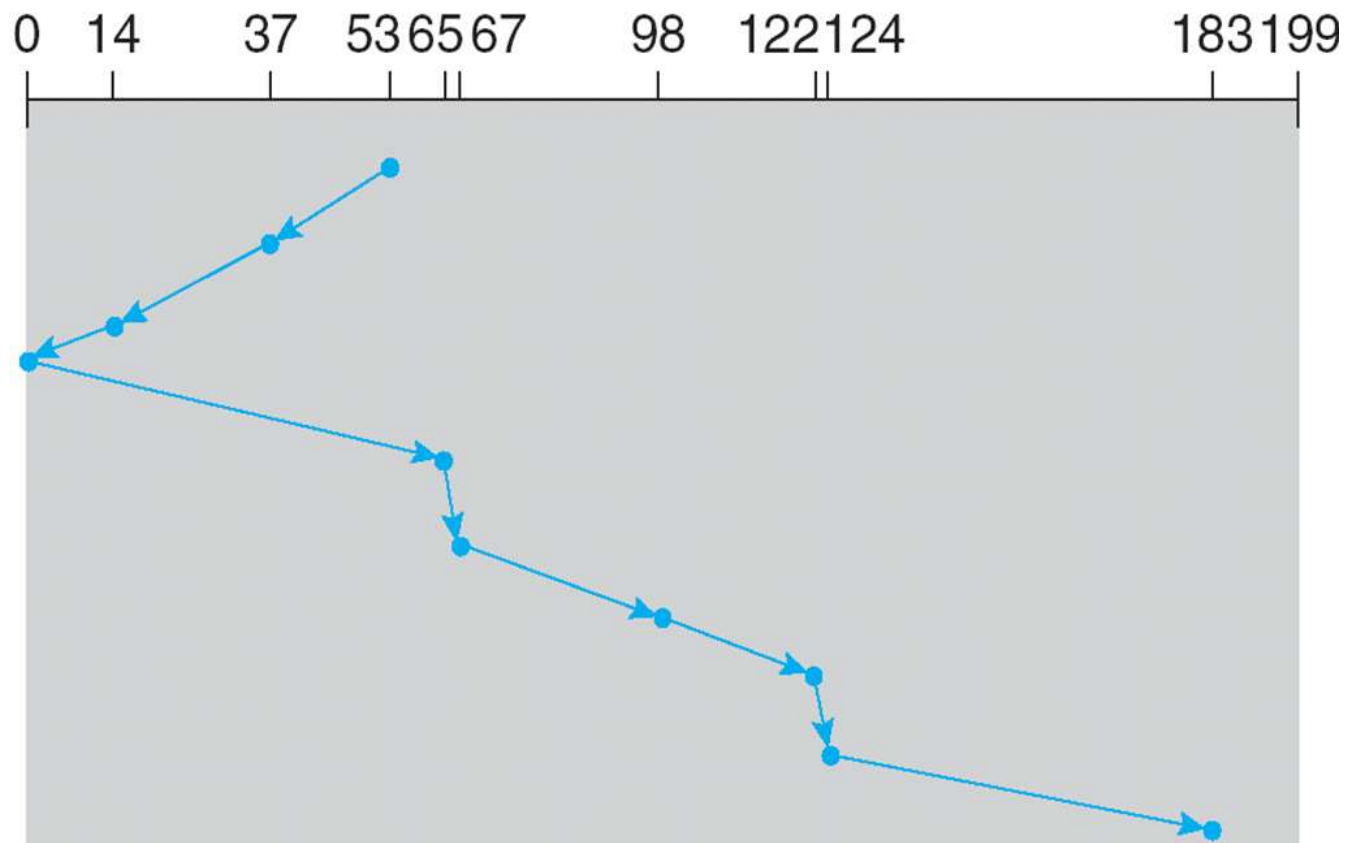
- La testina inizia ad un estremo del disco e move verso l'altro estremo servendo le richieste finché non arriva all'altro estremo dove inverte il percorso
- **SCAN algorithm** chiamato anche **elevator algorithm**
- Nota che se le richieste non sono uniformemente dense con densità accumulate all'altro estremo del disco, queste possono aspettare molto

SCAN

- La figura mostra un movimento totale di 236 cilindri
- Nota che se le richieste non sono uniformemente dense con densità accumulate all'altro estremo del disco, queste aspettano molto

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53

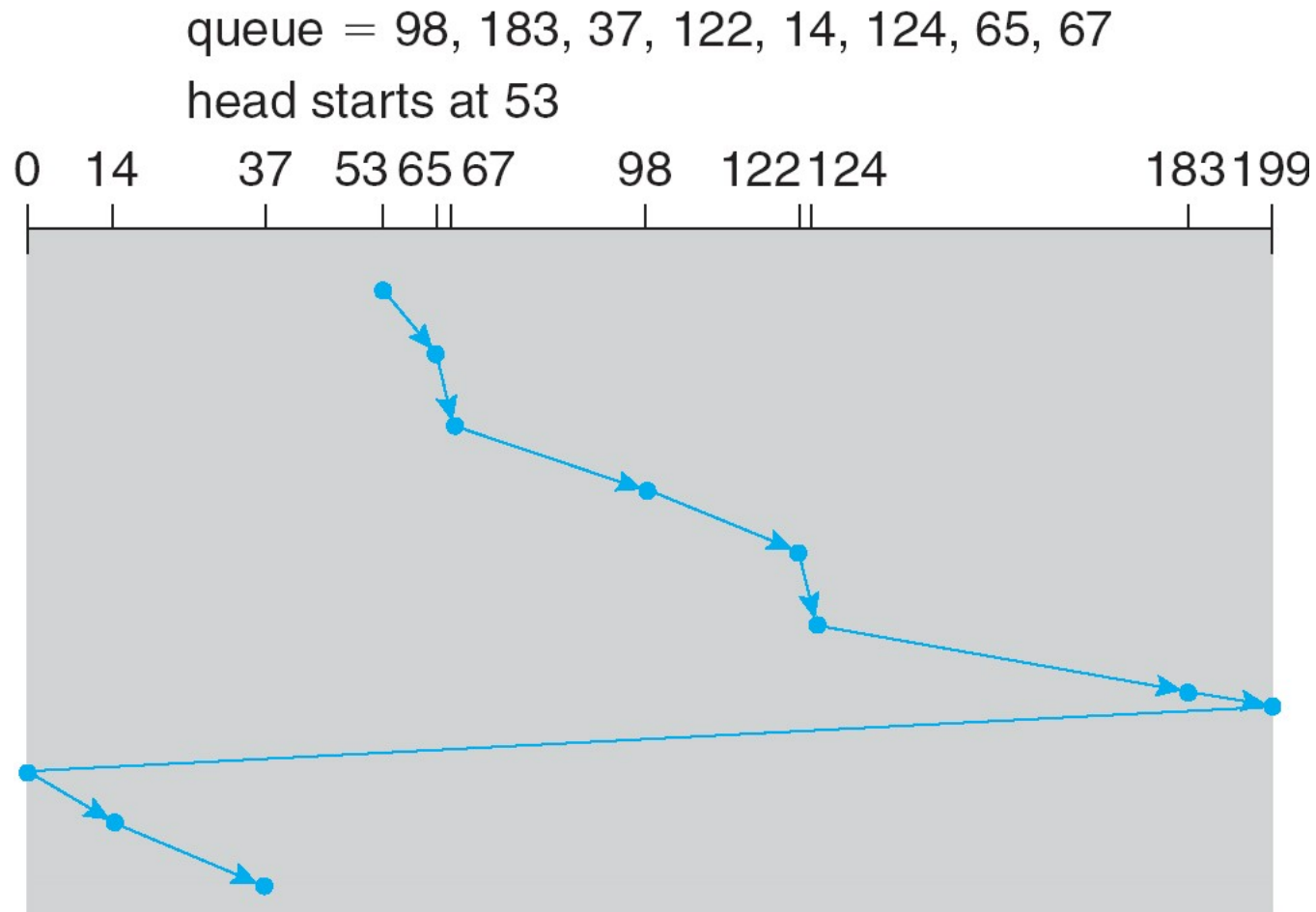


C-SCAN

- Tempi di attesa più uniformi rispetto a SCAN
- La testina si muove da un estremo del disco all'altro servendo le richieste nel mentre
 - Quando raggiunge l'altro estremo, comunque, immediatamente torna all'inizio del disco, senza servire le richieste nel viaggio di ritorno
 - Costo del ritorno ...
 - ... ma le richieste sono più frequenti nei cilindri esterni
- Mette i cilindri in una lista circolare dove l'ultimo cilindro porta al primo

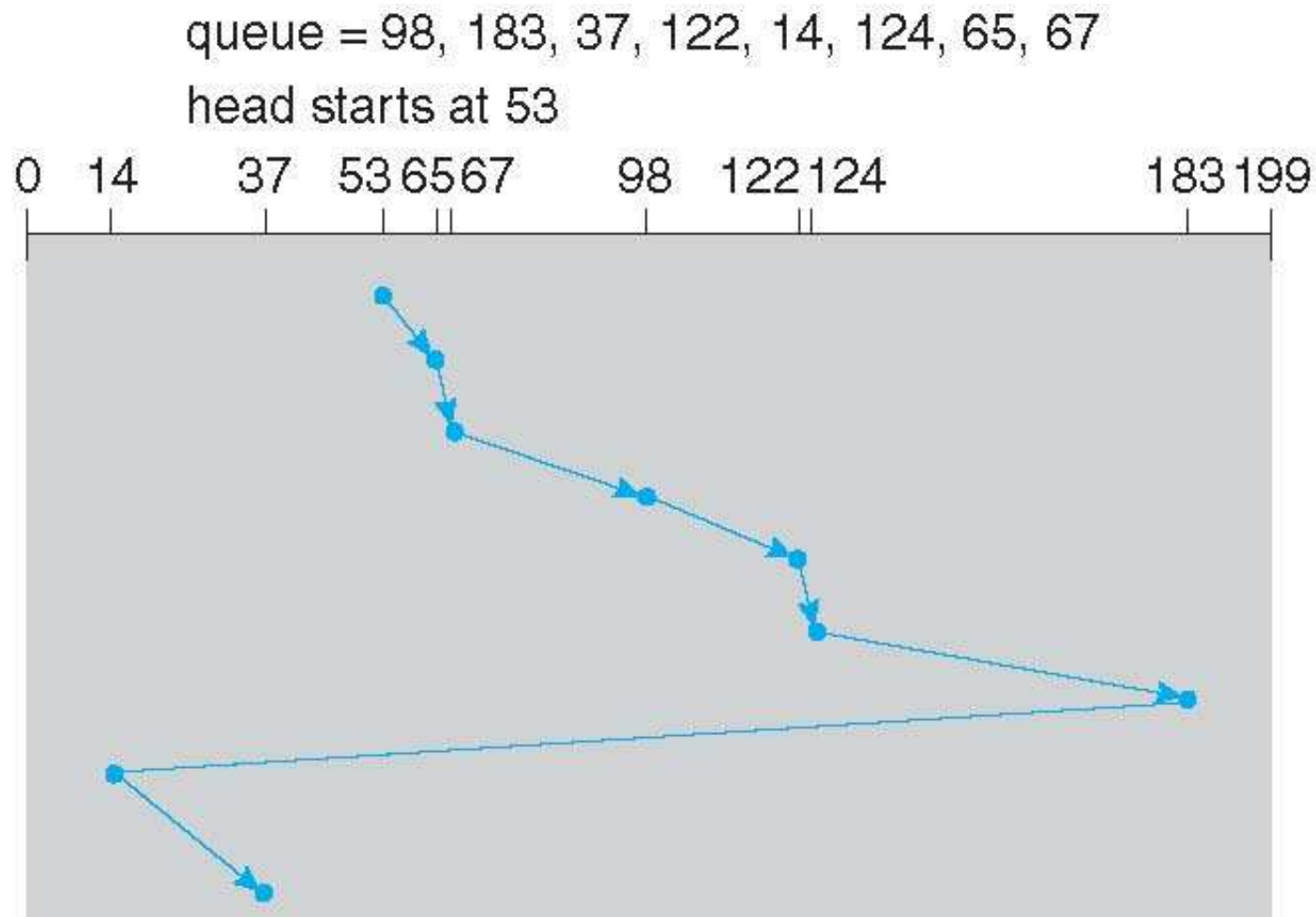
C-SCAN

- Si assume il movimento sempre verso destra



C-LOOK

- LOOK è un tipo di SCAN, C-LOOK è una versione di C-SCAN
- Il braccio avanza fino all'ultima richiesta in ogni direzione, poi cambia direzione

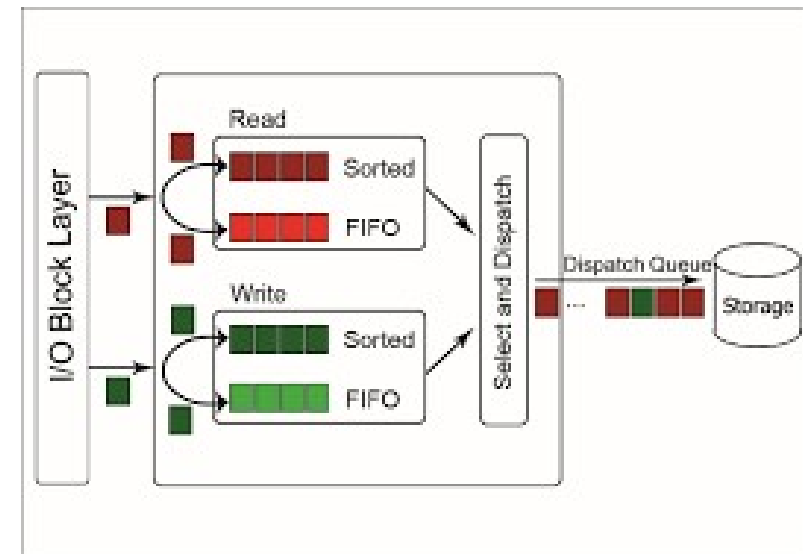


Selezionare un Algoritmo di Disk-Scheduling

- SSTF (Shortest Seek Time First) è tipico e naturale
- SCAN e C-SCAN funzionano meglio per sistemi con grosso carico su disco
 - Meno starvation
- Performance dipende dal numero e dai tipi di richiesta
- Le richieste di servizio possono essere influenzate dai metodi di file-allocation
 - File allocato in modo contiguo meno movimenti di testina
 - File likato sparso più movimenti di testina
 - Allocazione delle directory (file aperti richiedono ricerca nella struttura)
- Algoritmi di disk-scheduling implementati come moduli separati del SO per consentire un aggiornamento se necessario
- SSTF che LOOK sono una ragionevole algoritmo di default

Selezionare un Algoritmo di Disk-Scheduling

- In alcuni sistemi più code con priorità per evitare starvation
- Linux – deadline scheduler con code di read e write con priorità e ordine LBA (Logical Block Addressing) implementate in modo simile a C-SCAN
 - read maggiore priorità perché bloccante
- Quattro code per gestire le deadline
 - 2 per read e 2 per write, una gestita con LBA l'altra con FCFS
 - Le richieste gestite in batch, dopo ogni batch si controlla se ci sono richieste in FCFS vecchie (più di 500 ms per read e 5 sec per write) e si sceglie quello come prossimo batch LBA
- Quando scade il timer di deadline, Deadline comincia a servire le richieste dalla coda FIFO. Si serve per ordine di arrivo. Si contrasta la starvation delle richieste (con preferenza alle richieste in lettura)



Selezionare un Algoritmo di Disk-Scheduling

- ❑ Lo scheduler NOOP (NO OPeration) è il più semplice fornito da Linux.
- ❑ `echo noop > /sys/block/sda/queue/scheduler`
- ❑ Funzionamento. Una coda di richieste, gestita in modalità FIFO con merging delle richieste contigue
- ❑ **Vantaggi.** Scheduler semplicissimo e molto efficiente in termini di esecuzione. Non ha nessuna logica di riduzione dei seek (a parte il merging) ideale per dispositivi non rotazionali → (SSD).
- ❑ **Svantaggi.** Non ha nessuna logica di riduzione dei seek (a parte il merging) disastroso sui dispositivi rotazionali. → Starvation delle richieste. Incapacità di differenziare in base all'importanza del processo.

NVM scheduling

- Lo scheduling HDD deve minimizzare i movimenti meccanici
- Nel caso di NVM non ci sono movimenti meccanici e si usa FCFS
 - Es. In Linux NOOP usa FCFS modificato per servire richieste adiacenti
- Le richieste read sono servite in modo uniforme, ma write in modo non uniforme
 - Alcuni SO servono le read con FCFS, e accorpendo solo richieste di write
- I/O possono avvenire in modo random o sequenziale
 - Per HDD meglio sequenziale
 - Per NVM random è ok
 - input/output operations per second (IOPS) diversi per HDD e NVM
 - ▶ Nel caso di accesso random centinaia vs centinaia di migliaia
 - ▶ Nel caso di accesso sequenziale più simile
 - ▶ Le performance di NVM degradano nel tempo e la scrittura è più lenta della lettura

Gestione del Disco

- Il Sistema Operativo è responsabile di diverse operazioni
 - Inizializzazione, bootstrap, bad-block recovery

- Il dispositivo deve essere strutturato
 - HDD in tracce e settori, NMV con pagine e FTL (Flash Transaltion Layer)
 - **Formattazione di basso livello** o **fisca**, disco in settori che il disk controller può leggere e scrivere (produttore fornisce e fa testing)
 - ▶ Utility di formattazione effettuano solo una rinizializzazione
 - Ogni settore è un blocco di dati definita da un header, dati, più un trailer
 - ▶ L'header permette l'identificazione del settore
 - ▶ Il trailer contiene il checksum e un error correction code (**ECC**)

- Il SO deve anche avere una strutturazione del disco
 - **Partizione** del disco in uno o più gruppi di cilindri, ognuno trattato come un disco logico separato

Error Detection and Correction

- Occorrono tecniche per trovare e correggere errori
- **Parity Bit**
 - Esempio di metodo checksums che utilizza metodi aritmetici per controllare dati di lunghezza fissata:
 - ▶ Ogni byte ha associato un bit che controlla se il numero di 1 è pari (parity = 0) oppure è dispari (parity = 1)
 - ▶ Se uno dei bit è danneggiato allora il parity non coincide più con quello precalcolato (incluso il danno dello stesso parity)
 - ▶ Tutti gli errori di un bit su un byte sono trovati (due bit non visti)
 - ▶ Il parity si calcola rapidamente con lo XOR ($1 \text{ xor } 1 = 0$)
- **Error Correction Code (ECC)** fa anche la correzione
 - Disk drive usano un ECC per settore, NVM per pagina
 - Quando si scrive un settore/pagina ECC è scritto
 - Quando si legge ECC è controllato, se non coincide allora il settore/pagina è bad
 - Se soft error può essere corretto (pochi bit) altrimenti hard error

Error Detection and Correction

- ❑ Occorrono tecniche per trovare e correggere error
- ❑ **Error Correction Code (ECC)** fa anche la correzione
 - ❑ Disk drive usano un ECC per settore, NVM per pagina
 - ❑ Quando si scrive un settore/pagina ECC è scritto
 - ❑ Quando si legge ECC è controllato, se non coincide allora il settore/pagina è bad
 - ❑ Se soft error può essere corretto (pochi bit) altrimenti hard error
- ❑ Esempio ECC (Hamming)
 - ▶ k bit + r di parità ($r \geq \log_2(m + 1)$ con m bit totale in codice)
 - ▶ Es. Dati D1,D2,D3,D4 e parity P1, P2, P3
 - ▶ $P1 = \text{Parity}(1,2,4)$, $P2 = \text{Parity}(1,3,4)$, $P3 = \text{Parity}(2,3,4)$
 - ▶ Se non parity errore 1,0,1,1,P1,P2,P4 sarà 0 1 0
 - ▶ Se non torna la parità errore e possibilità di individuarlo

Gestione dei Bad Block

- ❑ Blocchi del disco possono essere corrotti
- ❑ Nei vecchi sistemi la ricerca dei bad blocchi richiedeva una scansione lanciata dall'utente
- ❑ Una gestione più sofisticata è fornita dai controllori del disco più recenti
- ❑ Il controllore del disco mantiene una lista dei bad block del disco e ha a disposizione dei settori di ricambio non visti dal SO
 - ❑ Quando prova ad accedere ad un settore e lo trova corrotto (ECC)
 - ❑ Riporta l'errore all'SO e marca il settore come bad e lo sostituisce con un **settore di richambio**
 - ❑ Quando è richiesta il medesimo blocco logico, questo viene tradotto nel nuovo settore di ricambio
 - ❑ Sostituzione può interferire con lo scheduling del disco se le riparazioni sono "lontane", per questo i settori di ricambio si cercano nello stesso cilindro
 - ▶ Altro metodo è il sector slipping che fa slittare i settori
 - ▶ Nota: in NVM la gestione è più semplice, non c'è seek time da gestire

Gestione del Disco

- Il SO deve anche avere una strutturazione del disco



Gestione del Disco

- Il SO deve avere una strutturazione logica del disco
 - **Partizione** del disco in uno o più gruppi di cilindri, ognuno trattato come un disco logico separato (info su partizioni è su locazione del disco)
 - In Linux fdisk per avere info sul dispositivo di storage
 - Il Sistema Operativo riconosce un dispositivo e legge la sua info su partizione
 - Il Sistema crea una entry per quella partizione (in Linux /dev)
 - Per rendere un file system disponibile l'SO deve montare la partizione
- Il secondo step è la creazione e gestione di un **volume** (drive logico)
 - Creazione del volume può essere implicita o esplicita
 - Linux volume manager lvm2
- Il terzo step è la **formattazione logica** o la creazione di un file system
 - Per incrementare l'efficienza i file system raggruppano i blocchi in **clusters**
 - ▶ Disk I/O fatto in blocchi
 - ▶ File I/O fatto in clusters (insiemi di blocchi)
 - Partizione indica anche se la partizione consente il bootstrap

Gestione del Disco

- Esempio del tool di disk management di Windows 7
 - Illustrati tre volumi C: E: F:, E: F: in partizioni del disco 1, c'è spazio non allocato per più partizioni

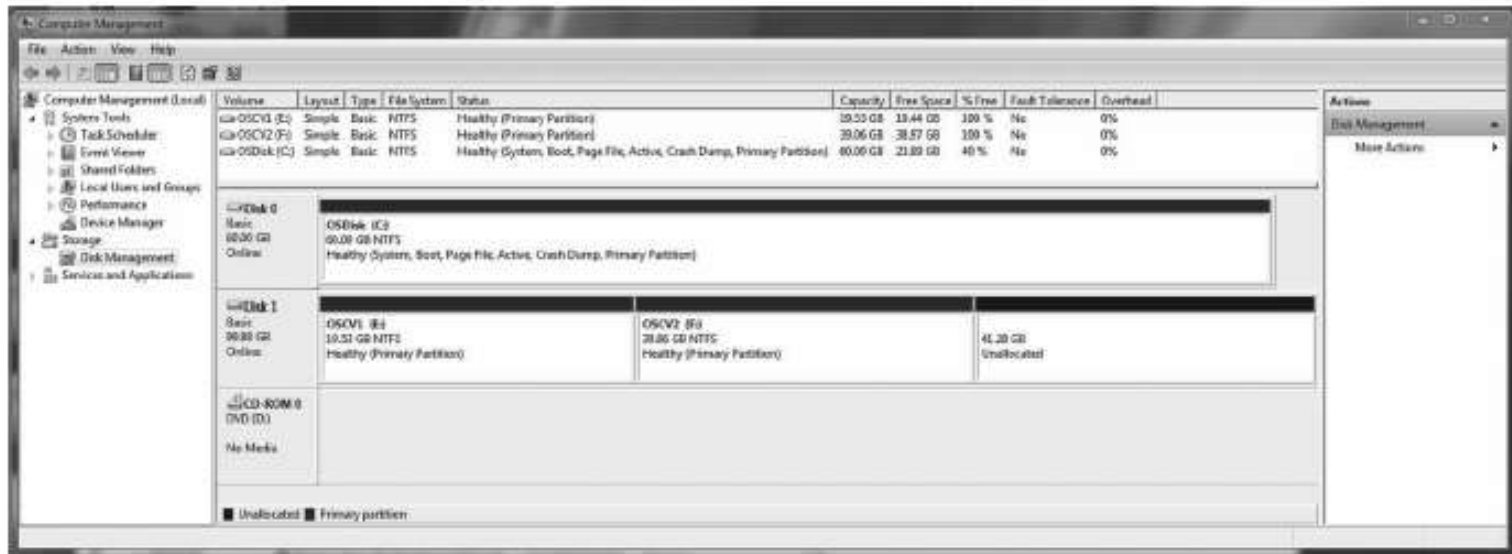
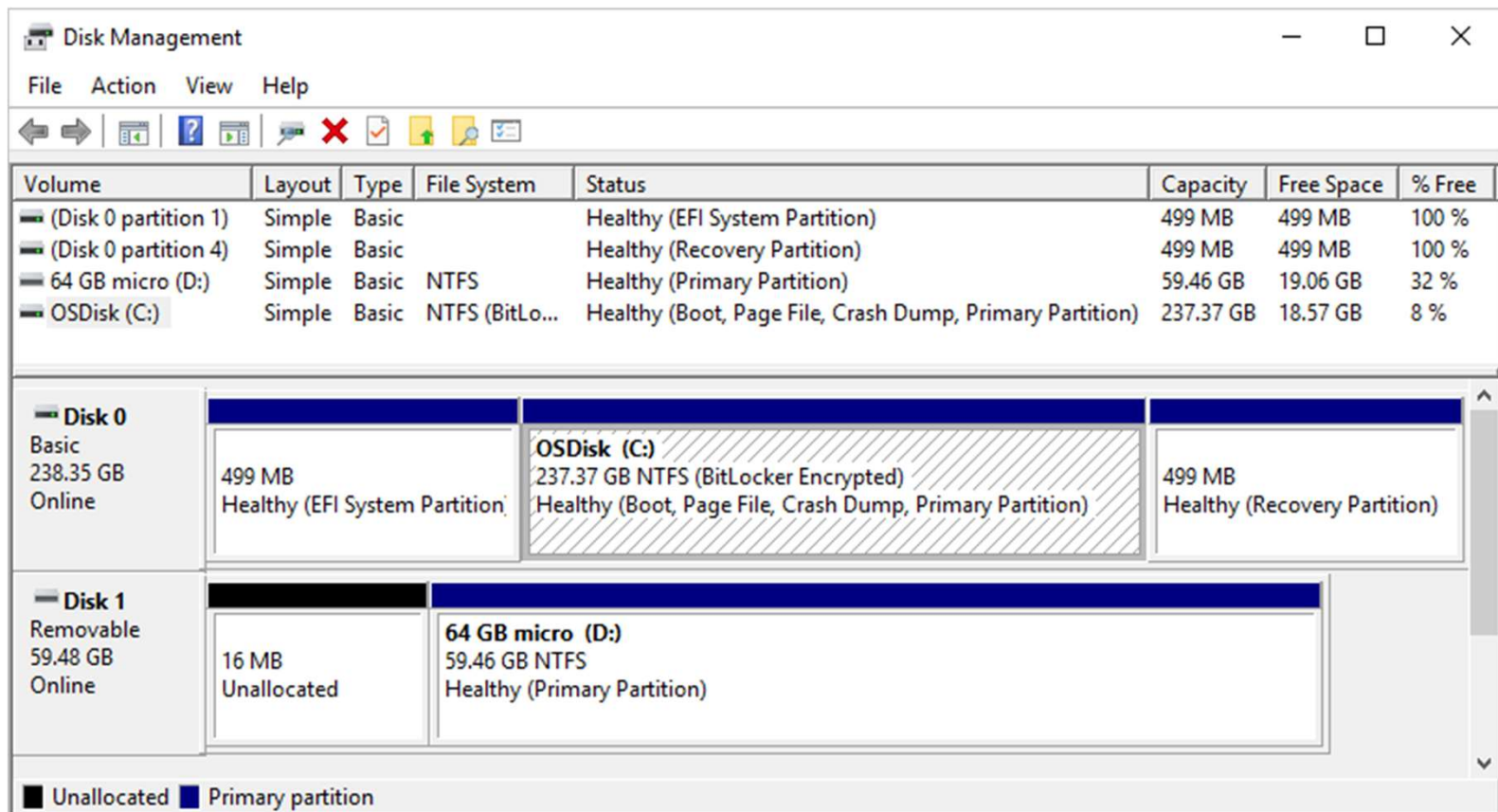


Figura 11.9 Lo strumento gestione dischi di Windows 7 mostra i dispositivi, le partizioni, i volumi e i file system.

Gestione del Disco

- Esempio del tool di disk management di Windows 10
 - Illustrati due volumi C: D:, C: in partizione del disco 0 D: in partizione disco 1, c'è spazio non allocato per più partizioni



Gestione del Disco

- Dal Boot block si inizializza il sistema
 - I primi passi del bootstrap sono in firmware (test e avvio del **boot**)
 - Avvio del processo di **boot gestito** con MBR (Master Boot Record) o GPT (GUID Partition Table)
 - Con MBR
 - ▶ 512 byte: boot code (es. 446 byte), tabella partizioni (es. 64 byte), check (2 byte)
 - ▶ Portato in memoria boot code dal Master Boot Record
 - ▶ Può leggere partizioni e cercare i boot blocks
 - ▶ Boot Code carica un bootloader per fare il bootstrap
 - ▶ Il **bootloader program** è nel “boot block”
 - Un dispositivo che ha il boot block è un boot disk o system disk
 - ▶ Con il caricamento del boot block si può lanciare il bootloader program che poi caricherà il Kernel (bootstrap)

Gestione del Disco

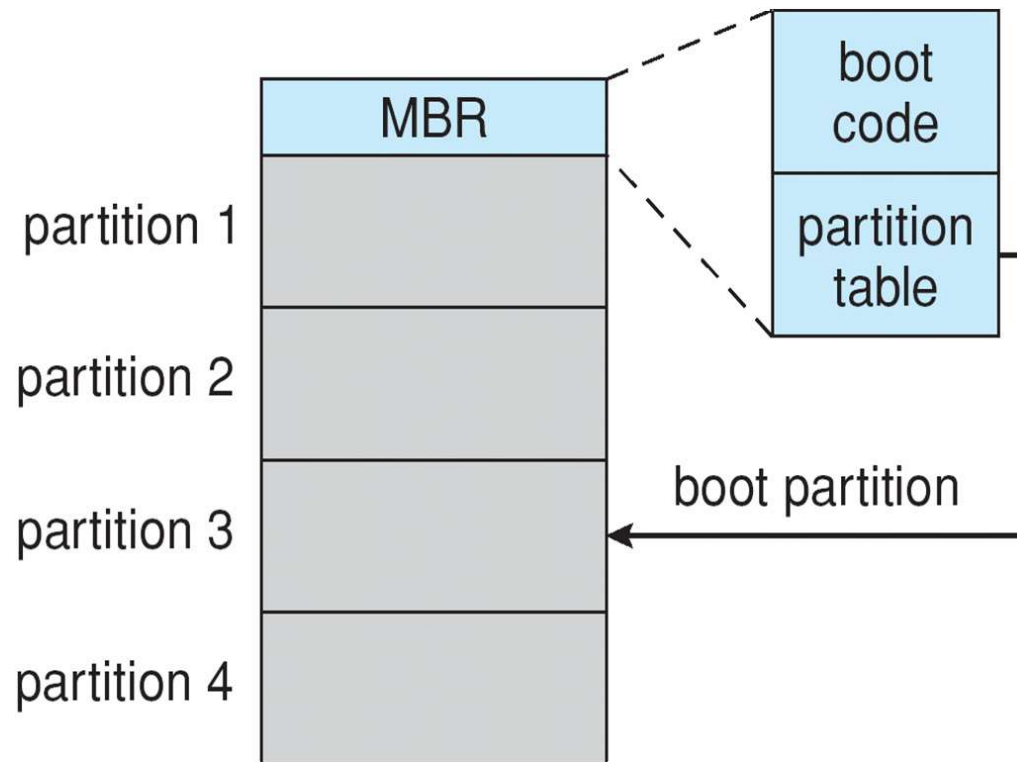
- Dal Boot block si inizializza il sistema
 - Esempio Windows (con BIOS - Basic Input/Output System)
 - ▶ Boot inizia lanciando il programma nel firmware (test e avvio)
 - ▶ Test POST (Power-On Self Test)
 - ▶ Programma avvio nel primo blocco logico di HDD o prima pagina di NVM - Master Boot Record (MBR)
 - ▶ MBR ha codice di avvio (boot code) e tabella delle partizioni, da queste si risale alle partizioni di boot dove trova il bootloader NTLDR
 - ▶ NTLDR carica il kernel

Booting da disco in Windows

Esempio Windows (BIOS - Basic Input/Output System)

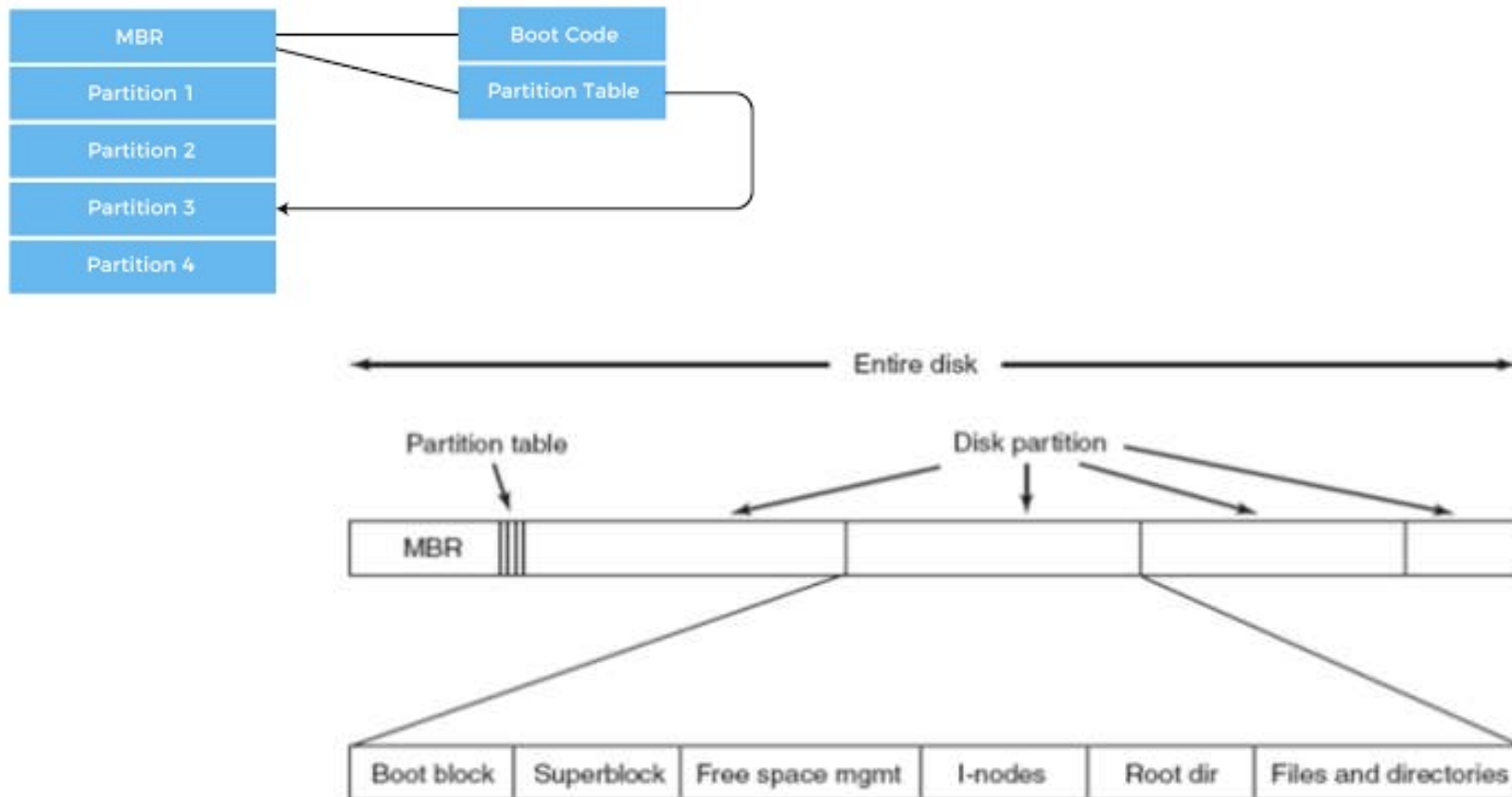
- Firmware all'avvio test e lancio del boot
- *boot code* nel Master Boot Record (MBR)
- Il firmware carica il boot code dal MBR (lancia il sistema)
- MBR ha boot code e tabella delle partizioni, da queste si risale alle partizioni di boot dove trova il bootloader

MBR - settore 512-byte
con settori di 512
 2^{32} indirizzi
Indirizzabili 2.2TB



Partizioni

- Il boot block può puntare a un volume di boot
 - Il codice nel boot block consente di caricare il Sistema
 - Il superblock contiene parametri del file system

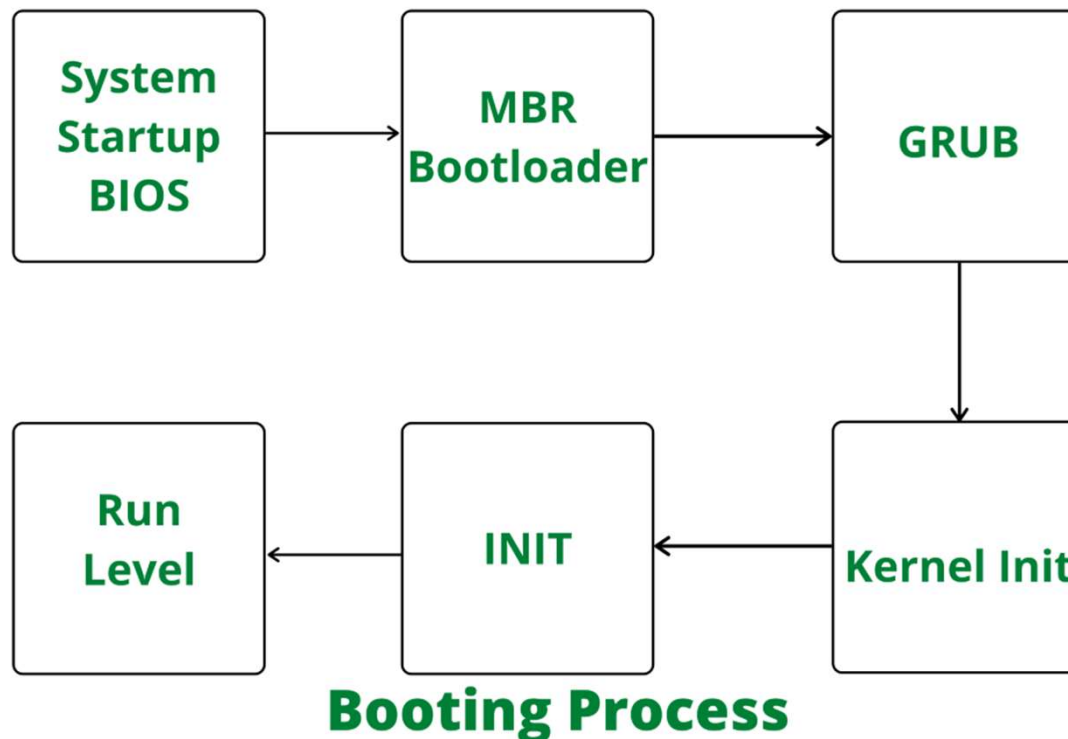


A possible file system layout.

Booting da disco in Linux

Esempio Linux (con BIOS - Basic Input/Output System)

- Firmware all'avvio test e lancio del boot code (MBR bootloader)
- Bootloader di secondo livello (Grub2, systemd-boot)
- Il bootloader presenta un menu di possibili sistemi
- Una volta selezionato il sistema viene caricato il kernel in memoria
- Chiamato lo `start_kernel()` che lancia `idle_process`, `scheduler`, `init`

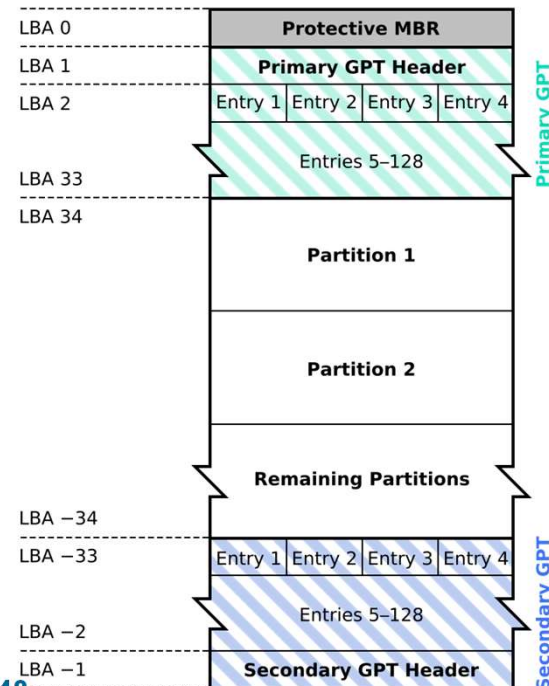


Booting da disco GPT

Boot con UEFI e disco GPT (GUID Partition Table)

- Usa identificatori univoci globali (GUID) per le partizioni
- Supporta dischi con capacità superiore a **2 TB** e fino a **128 partizioni**
- UEFI (Unified Extensible Firmware Interface) sostituisce BIOS
- Non usa MBR ma partizione ESP (EFI System Partition) nel disco GPT
- La GPT occupa i primi settori del disco:
 - LBA 0: Settore compatibilità (MBR protettivo) per indicare disco GPT
 - LBA 1: Intestazione GPT primaria, descrive la struttura delle partizioni
 - LBA 2 e succ.: Tabella partizioni GPT (entry per ogni partizione, inclusa ESP)

GUID Partition Table Scheme



Booting da disco GPT

Boot con UEFI e disco GPT (GUID Partition Table)

- Usa identificatori univoci globali (GUID) per le partizioni
- Supporta dischi con capacità superiore a **2 TB** e fino a **128 partizioni**
- UEFI (Unified Extensible Firmware Interface) sostituisce BIOS
- Non usa MBR ma partizione ESP (EFI System Partition) nel disco GPT
- La GPT occupa i primi settori del disco:
 - LBA 0: Settore compatibilità (MBR protettivo) per indicare disco GPT
 - LBA 1: Intestazione GPT primaria, descrive la struttura delle partizioni
 - LBA 2 e succ.: Tabella partizioni GPT (entry per ogni partizione, inclusa ESP)
- Firmware UEFI all'avvio test - POST (Power-On Self Test)
- Il firmware consulta GPT e cerca file di avvio in ESP
- Il firmware carica il Boot Manager UEFI da ESP
- Il **Boot Manager UEFI** legge configurazioni di avvio e gestisce caricamento del kernel
- Può gestire diversi SO:
 - Ogni SO installato crea una directory nella ESP con i suoi file di avvio
 - Win /EFI/Microsoft/Boot/bootmgfw.efi
 - Linux /EFI/Ubuntu/grubx64.efi
 - macOS /EFI/APPLE/BOOT/BOOTX64.EFI

Partizioni e Montaggio

- ❑ Partizione può essere un volume che contiene un file system (“cotta”) or **raw** (cruda)– una sequenza di blocchi senza un file system
- ❑ Il boot block può puntare a un volume di boot; il bootstrap loader carica codice per caricare il kernel dal file system
 - ❑ ... o un programma boot management per boot con multipli SO
 - ❑ Il boot loader deve interpretare il formato del FS per poter caricare i blocchi
- ❑ **Partizione root** contiene il SO, altre partizioni possono contenere altri sistemi, altri file system, o possono essere raw
 - ❑ Montata a tempo di boot
 - ❑ Altre partizioni si possono montare automaticamente o manualmente
 - ❑ Su win montato su volume differenti name-space (lettere F:, E:, etc.)
- ❑ A tempo di mount, viene controllata la consistenza del file system
 - ❑ I metadata sono corretti?
 - ▶ se non lo sono, aggiusta e riprova
 - ▶ se lo sono, aggiungi la tabella del montaggio, permetti l'accesso

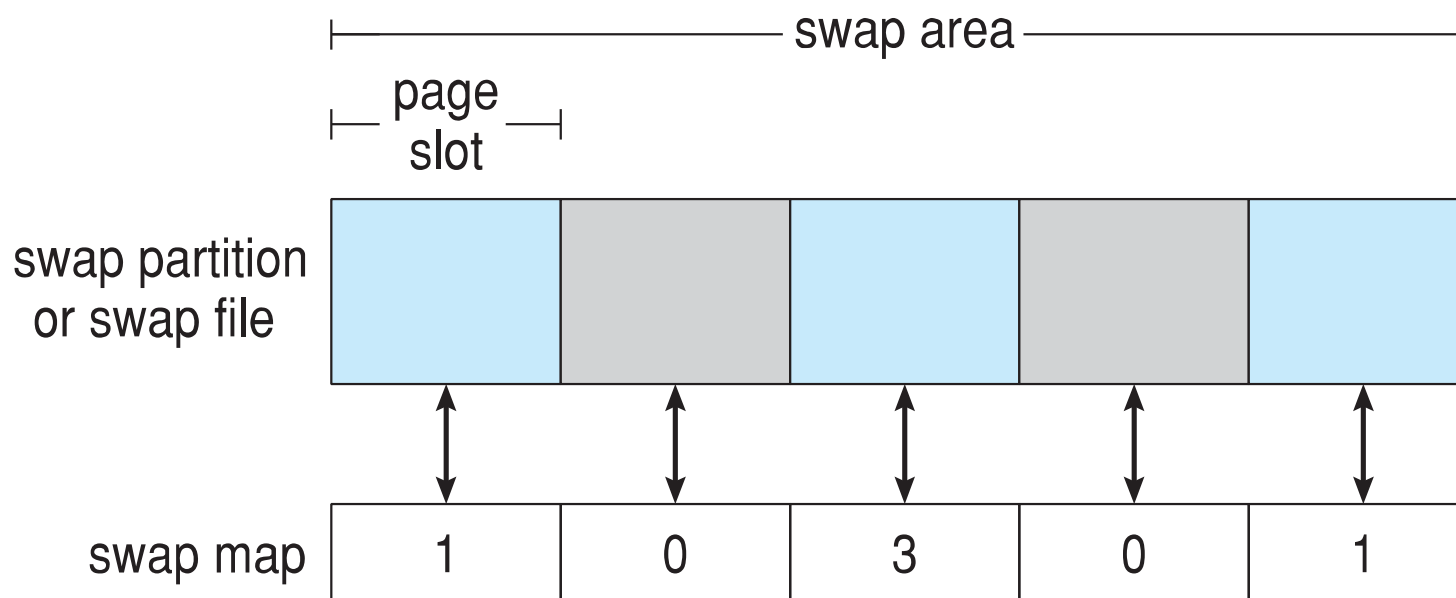
Gestione dello Swap-Space

- Swap-space — la memoria virtuale usa spazio su disco come estensione della memoria principale
- Lo swap space può variare da pochi megabytes a gigabytes a seconda della dimensione della memoria fisica, di quella virtuale e da uso
 - Solitamente meglio sovrastimare che sottostimare perché se finisce i processi possono essere abortiti o arrivare al crash di Sistema
 - Es. Solaris suggerisce tanto quanto $SS = VM - FM$, Linux prima $SS = FM$ ora meno
 - In Linux multipli swap space su file system o su diverse partizioni
- Swap-space può essere ottenuto dal normale file system, ma più comunemente da una partizione del disco separata (raw)
 - Le partizioni raw permettono accesso più veloce ma è meno efficiente lo stoccaggio (frammentazione interna), cmq dati in swap a breve termine
 - Linux permette swap sia su file system che su partizione raw

Gestione dello Swap-Space

- Gestione dello Swap-space – Esempio nei Sistemi UNIX
 - Nei primi sistemi il Kernel faceva swapping dell'intero processo in locazioni contigue del disco, poi si è passata alla gestione con paging
 - In Solaris 1 si rileggono le pagine di text direttamente dal file system, memoria anonima su swap space (stack, heap, memoria non inizializzata)
 - Solaris 2 alloca swap space solo quando una dirty page viene mandata fuori dalla memoria fisica (non quando è creata la pagina della virtual memory)
 - In Linux, come in Solaris, swap space solo per memoria anonima
 - ▶ Mantiene più aree swap (sia file system sia raw) – area composta di pagine di 4 KB
 - ▶ Il Kernel usa **mappe di swap** per tracciare l'uso dello swap-space – array di contatori interi che contano quanti processi si riferiscono a quella pagina (es. 0 pagina libera, 3 indica tre processi che condividono la stessa pagina)

Strutture Dati Kernel per lo Swapping in Linux



Occorre una struttura che fornisca informazioni sulla memoria di swap (`swap_info_struct` in linux): dispositivo, blocchi usati, liberi, etc.

Occorre anche la mappatura tra (pagine,pid) e blocchi in memoria di swap

Struttura RAID

- RAID – Redundant Array of Inexpensive Disks
 - Multiple unità disco forniscono affidabilità e performance con la **ridondanza**
 - Prima Inexpensive ora Independent
- La probabilità di fallimento di N dischi è più bassa
- Consideriamo il **mean time between failure (MTBF)**
 - Se 100.000 ore per un disco, in un array di 100 dischi il MTBF di almeno un disco è 1000 ore, cioè 41,66 giorni ...
 - Senza replicazione dei dati è un problema
- Modo più semplice di replicare è il **mirroring** duplicazione dei dischi
 - Ogni disco logico corrisponde a più dischi fisici (es. scrittura doppia)
 - Perdita di dato solo se anche il secondo disco fallisce (prima della riparazione del primo)

Mirrored Disk

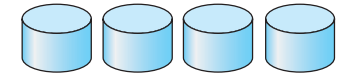
- Mirrored Disk
 - Oltre al **mean time between failures**
 - **Mean time to repair** – tempo di esposizione quando un altro fallimento potrebbe causare perdita di dati
 - **Mean time to data loss** tempo medio di perdita dei dati dovuti a fallimento
 - Se i mirrored disks fallissero indipendentemente, preso un disco con 100000 ore di mean time between failures e 10 ore di mean time to repair
 - Il mean time to data loss è 57000 anni!
- Però i fallimenti non sono indipendenti
 - Problemi di alimentazioni particolarmente problematici durante la scrittura
 - Scrivere in momenti diversi le repliche, oppure usare NVM come cache

Disk Striping

- Nel mirroring duplicate le scritture, però le richieste di lettura solo ad una delle unità e questo non migliora il trasferimento
- Disk **striping** usa un gruppo di dischi come un'unica unità di storage
 - Dati distribuiti (strisciati) su più dischi
 - **Bit-level striping**, ogni i -esimo bit sul drive i -esimo
 - **Block-level striping**, blocco i -esimo su $(i \bmod n) + 1$ drive
 - ▶ Tipicamente questo è il metodo
- Due obiettivi principali
 - Incrementare il throughput di piccoli accessi bilanciando il carico
 - Ridurre il tempo di risposta di un accesso grande

Redundant Array of Inexpensive Disks

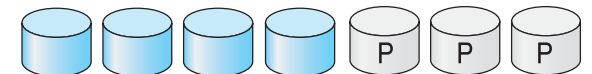
- Mirroring porta ridondanza, ma è costoso
- Striping amenta il trasferimento di dati, ma non l'affidabilità
- Alcuni schemi combinano striping con parity bit
- Organizzazioni a livelli RAID
- Es. 4 dischi per i dati, gli altri per ridondanza
 - Livello 0 non ridondanza, molto fragile
 - ▶ No fault tolerance: se un disco fallisce dati persi
 - ▶ Veloce, parallelismo, no parity control
 - ▶ Minimo numero di dischi 2
 - Livello 1 mirrored (C copia)
 - ▶ Dati duplicati, non overhead in velocità di scrittura
 - ▶ Velocizza la lettura (dati da più dischi)
 - ▶ Metà della capacità del disco
 - ▶ Minimo numero di dischi 2



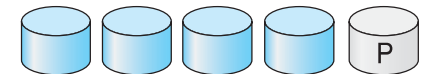
(a) RAID 0: non-redundant striping.



(b) RAID 1: mirrored disks.



(c) RAID 2: memory-style error-correcting codes.



(d) RAID 3: bit-interleaved parity.



(e) RAID 4: block-interleaved parity.



(f) RAID 5: block-interleaved distributed parity.



(g) RAID 6: P + Q redundancy.

RAID 2, 3, 4

❑ RAID 2 (poco utilizzato)

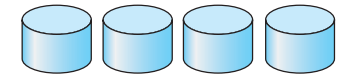
- ❑ Utilizza striping al livello di bit
- ❑ Utilizza hamming code per error detection/correction
- ❑ Minimo numero di dischi 3

❑ RAID 3 (poco utilizzato)

- ❑ Utilizza byte level striping
- ❑ Disco con un parity disk
- ❑ Minimo numero dischi 3

❑ RAID 4

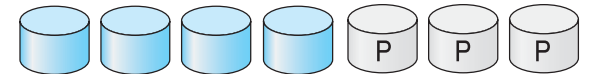
- ❑ Utilizza block level striping
- ❑ Disco con parity disk
- ❑ Scrittura lenta, scrittura del parity su unico disco
- ❑ Minimo numero dischi 3



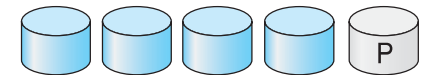
(a) RAID 0: non-redundant striping.



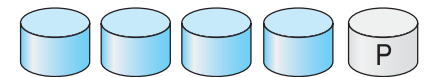
(b) RAID 1: mirrored disks.



(c) RAID 2: memory-style error-correcting codes.



(d) RAID 3: bit-interleaved parity.



(e) RAID 4: block-interleaved parity.



(f) RAID 5: block-interleaved distributed parity.



(g) RAID 6: P + Q redundancy.

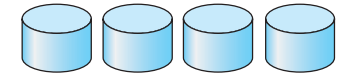
RAID 5, 6

- ❑ RAID 5 distribuisce i parity su tutti i dischi, evita di usare troppo il disco parity

- ❑ Minimo numero di dischi 3
- ❑ Block level striping + ECC distribuito
- ❑ ECC usato con stripping
 - ▶ Il primo blocco in drive 1, scondo in drive 2, N in drive N, error in $N + 1$
- ❑ Permette anche la correzione dei dati
- ❑ Più veloce del livello 1 nell'accesso
- ❑ Scrittura migliore di livello 4

- ❑ RAID 6 simile

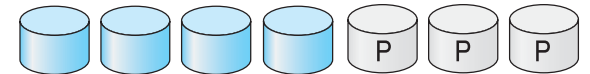
- ❑ Minimo numero di dischi 4
- ❑ aggiunge ridondanza per permettere recovery da fallimenti multipli
- ❑ 2 blocchi di parity distributi nei dischi
- ❑ Tollera due fallimenti di dischi
- ❑ Penalizzazione in scrittura



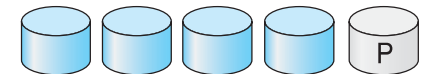
(a) RAID 0: non-redundant striping.



(b) RAID 1: mirrored disks.



(c) RAID 2: memory-style error-correcting codes.



(d) RAID 3: bit-interleaved parity.



(e) RAID 4: block-interleaved parity.



(f) RAID 5: block-interleaved distributed parity.



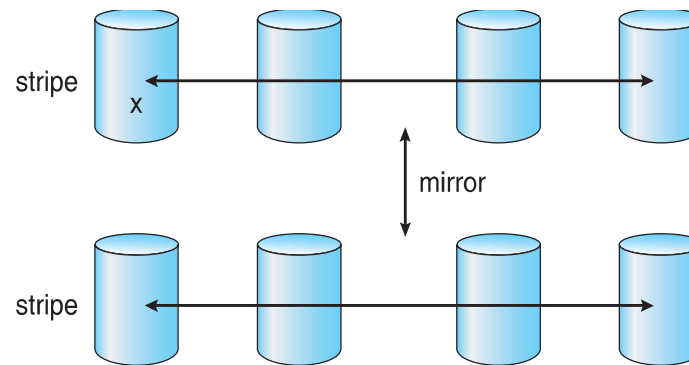
(g) RAID 6: P + Q redundancy.

RAID (0 + 1) e (1 + 0)

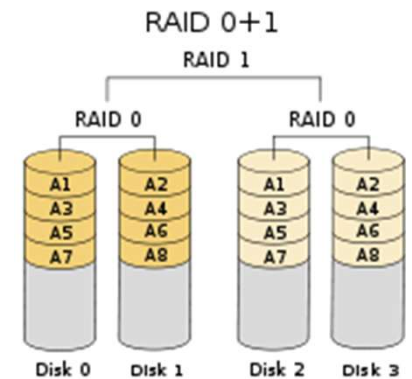
Nested RAID o Hybrid RAID combinano i livelli standard

Striped mirrors (**RAID 1+0**) o mirrored stripes (**RAID 0+1**) forniscono alta performance e alta affidabilità: RAID 0 performance, RAID 1 affidabilità

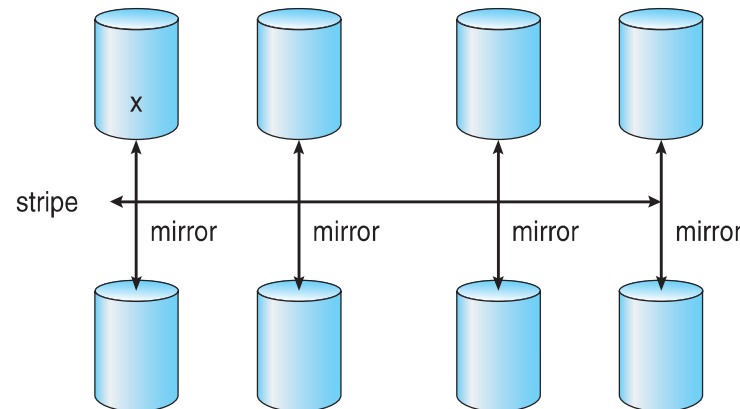
RAID 01 (0 + 1)
Dati striped e duplicate



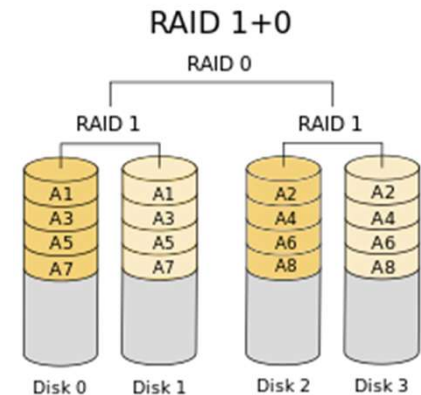
a) RAID 0 + 1 with a single disk failure.



RAID 10 (1 + 0)
Prima dati duplicati
poi striped sui dischi
(almeno 4)
Vicino a RAID 0 per
performance (usato
per sistemi I/O intense)
più ridondanza



b) RAID 1 + 0 with a single disk failure.



RAID

- Mirroring porta ridondanza, ma costoso
- Striping migliora il trasferimento di dati, ma non l'affidabilità
- Alcuni schemi combinano striping con parity bit

- RAID è organizzato in sei livelli differenti
- Gli schemi RAID aumentano le performance e l'affidabilità dello storage memorizzando dati ridondanti
 - **Mirroring** o **shadowing** (**RAID 1**) mantiene un duplicato di ogni disco
 - Striped mirrors (**RAID 1+0**) o mirrored stripes (**RAID 0+1**) fornisce alta performance e alta affidabilità
 - **Block interleaved parity** (**RAID 4, 5, 6**) usa molta meno ridondanza

Altre Caratteristiche

- Indipendentemente da come il RAID è implementato si possono aggiungere altri strumenti
- **Snapshot**
 - vista del file system prima che occorra un insieme di cambiamenti (i.e., ad un certo istante di tempo)
- **Duplicazione automatica** delle scritture tra siti separati
 - Per ridondanza e disaster recovery
 - Può essere sincrona o asincrona
- **Dischi di hot spare** (di ricambio)
 - **non usati per dati, ma solo in caso di fallimento per replicare il disco fallito e ricostruire il RAID set se possibile**
 - Diminuisce il mean time to repair